COMP90042
Natural Language Processing

THE UNIVERSITY OF MELBOURNE

Lecture 15 & 16: Tuning

Dr. Caren Han

Semester 1, 2025
School of Computing and Information Systems
University of Melbourne

/ Lecture Plan

THE UNIVERSITY OF MELBOURNE

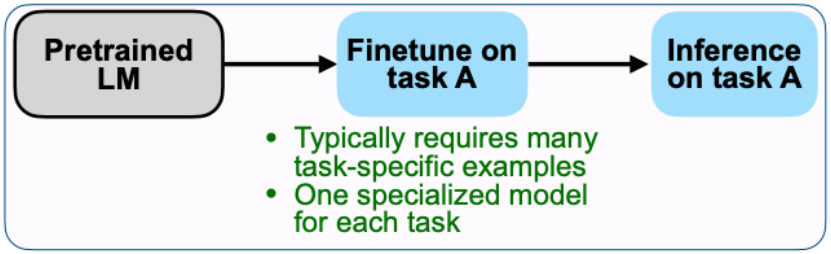
Lecture 15 &16: Tuning

0. **In-context Learning vs Fine-Tuning?**
0. **PLM vs LLM?**
1. Large Models and Tuning
2. Modular and Compositional Representations
3. Parameter Efficient Models

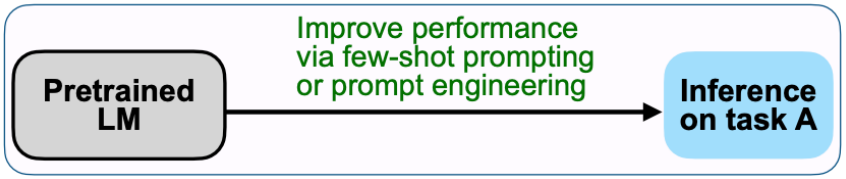
Recap: Prompting vs Instruction Tuning

(A) Pretrain-finetune VS (B) Prompting VS (C) Instruction Tuning

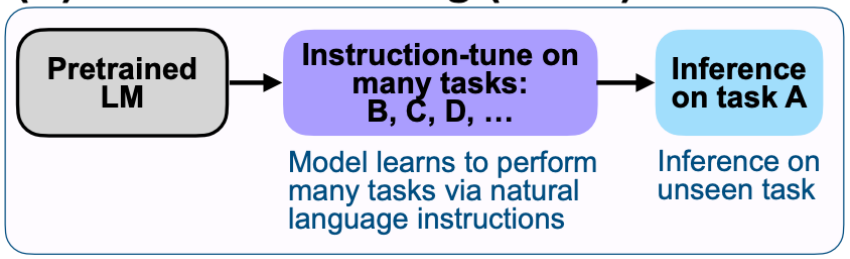
(A) Pretrain–finetune (BERT, T5)



(B) Prompting (GPT-3)



(C) Instruction tuning (FLAN)



0 Recap: Prompting vs Instruction Tuning

THE UNIVERSITY OF MELBOURNE

(B) Prompting VS (C) Instruction Tuning

Case: A user types, “Translate this sentence to Japanese: I love sushi,” into a model like GPT-3.5, which responds correctly without any fine-tuning.
Is this Prompting or Instruction Tuning?

✅ **Answer: Prompting (In-context learning)**

Case: Researchers fine-tune T5 on a dataset of task instructions (e.g., “Translate X to French”) with input–output pairs for dozens of NLP tasks.
Is this Prompting or Instruction Tuning?

✅ **Answer: Instruction Tuning**

Case: A prompt like “You are a helpful assistant. Answer concisely:” is prepended to every user query in ChatGPT to control tone and length.
Is this Prompting or Instruction Tuning?

✅ **Answer: Prompting (In-context learning)**

0 Recap: PLM vs LLM

THE UNIVERSITY OF MELBOURNE

Pretrained Language Model (PLM) **VS** Large Language Model (LLM)

Aspect	PLM	LLM
Scope	General term for pretrained language models	Subset of PLMs with large parameter count
Size	Can be small to medium	Usually very large (billions of params)
Architecture	Encoder, decoder, or both	Mostly decoder-only
Training Goal	Pretrain + fine-tune	General-purpose + prompt-based use
Usage Style	Fine-tuning-based	Prompt-based, few-shot or zero-shot
Examples	BERT, RoBERTa, T5	GPT-3/4, LLaMA

0 Recap: PLM vs LLM

THE UNIVERSITY OF MELBOURNE

Pretrained Language Model (PLM) **VS** Large Language Model (LLM)

1. Case: A company fine-tunes a BERT-based model to classify legal documents into predefined categories.

2. Case: A chatbot uses GPT-4 to answer open-ended questions by leveraging few-shot prompts.

/ Lecture Plan

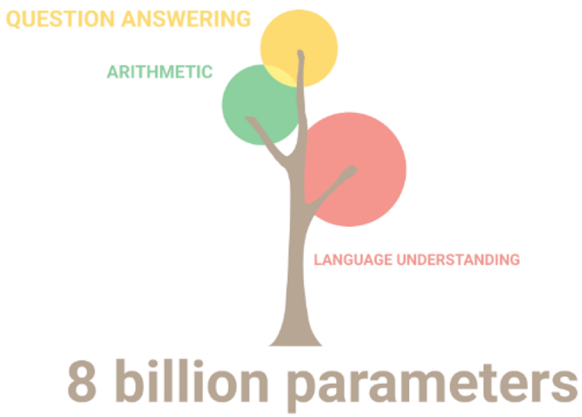
THE UNIVERSITY OF MELBOURNE

Lecture 15: Tuning

0. In-context Learning vs Fine-Tuning?
0. PLM vs LLM?
1. **Large Models and Tuning**
2. Modular and Compositional Representations
3. Parameter Efficient Models

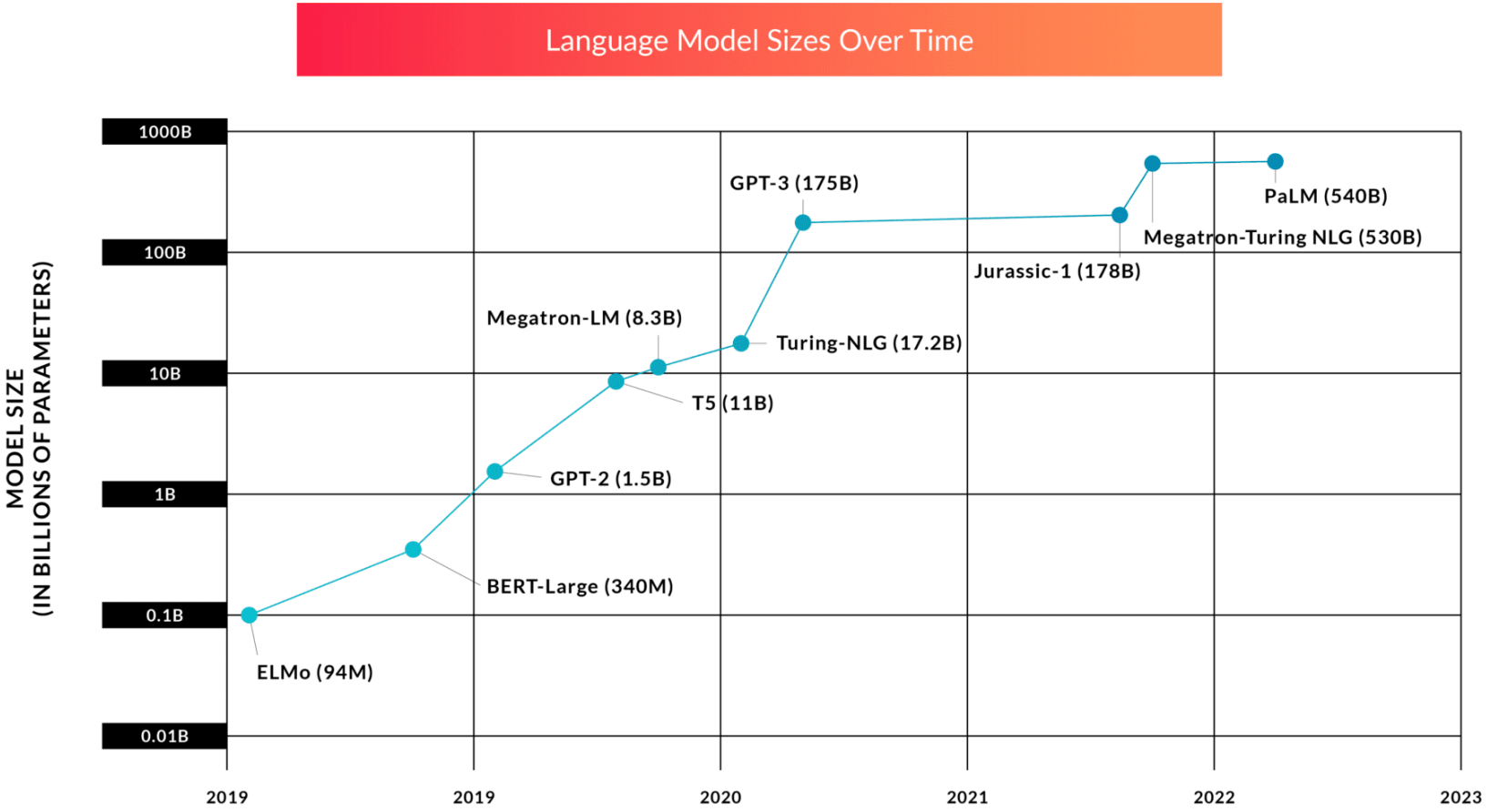
THE UNIVERSITY OF MELBOURNE

Large Models Develop Increasing Language Task Capabilities



THE UNIVERSITY OF MELBOURNE

State-of-the-art Language Models Are Getting Ever Larger



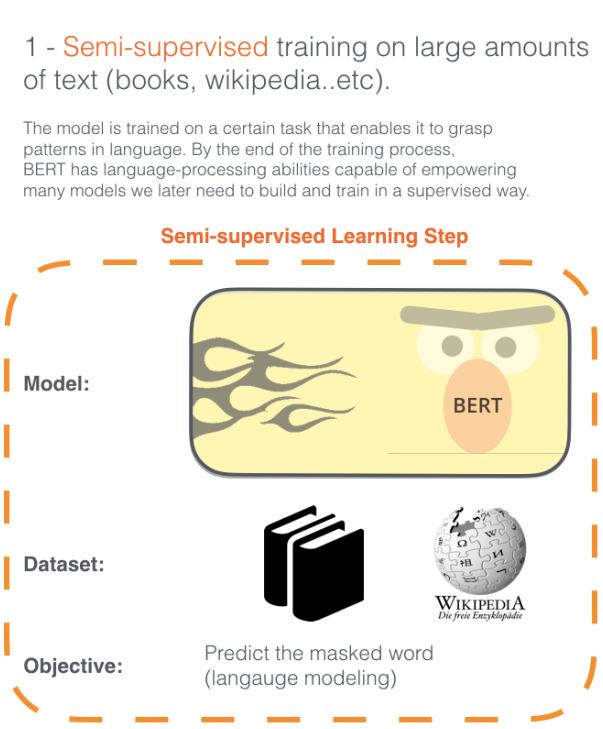
Large Models and Tuning

THE UNIVERSITY OF MELBOURNE

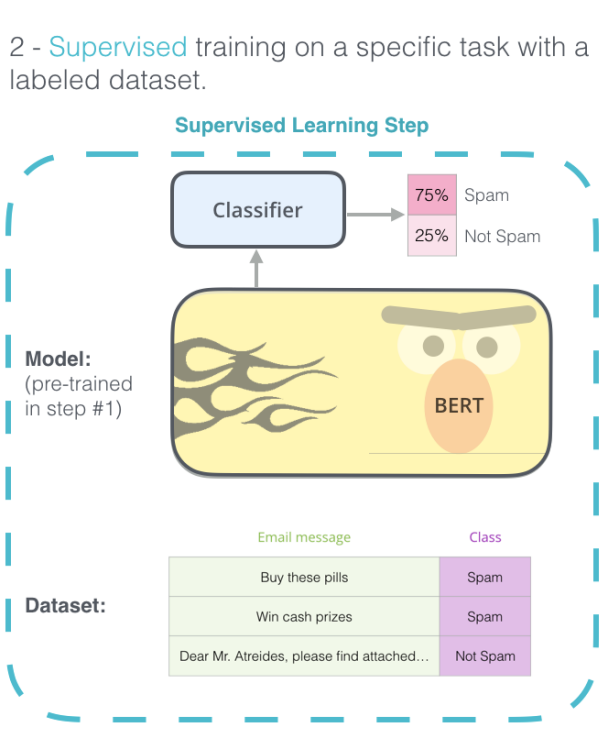
With increasing model size, fine-tuning becomes increasingly expensive

Step 1: Pretrain
(Lots of text; learn general things)

Step 2: Finetune (on **many tasks**)
(Not many labels; adapt to the task)



Pretraining



Fine-Tuning

If the model is too large, the standard transfer learning formula breaks down

- **Scalability issues**
  (e.g., huge compute requirements)
- **Instability** during training
  (e.g. overfitting, gradient interference)
- **Catastrophic forgetting**
  (losing general knowledge)

Large Models and Tuning

In-Context Learning Works really well!

- In-context learning has mostly replaced fine-tuning for large models

Zero-shot

The model predicts the answer given only a natural language description of the task. No gradient updates are performed.

```
Translate English to French:    ← task description
cheese =>                       ← prompt
```

Few-shot

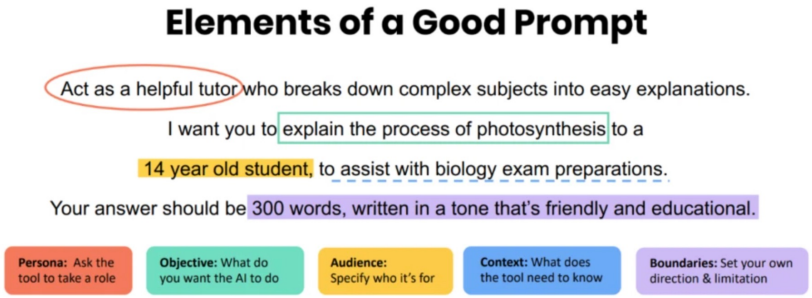
In addition to the task description, the model sees a few examples of the task. No gradient updates are performed.

```
Translate English to French:    ← task description
sea otter => loutre de mer      ← examples
peppermint => menthe poivrée    ←
plush girafe => girafe peluche  ←
cheese =>                       ← prompt
```

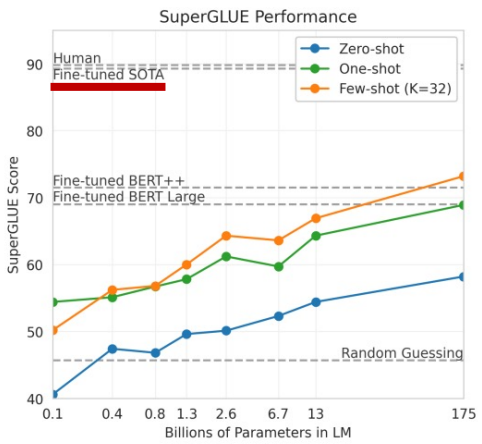
Large Models and Tuning

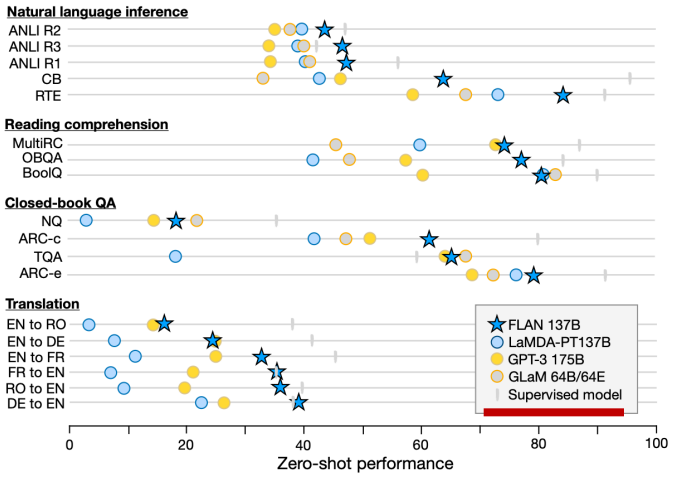
Reminder: Limitation of In-context Learning

1. Inefficiency: **Prompts** must be processed whenever the model makes a prediction.



2. Poor performance: **Prompting** generally performs **worse than fine-tuning**





(from lecture 13_14)

Large Models and Tuning

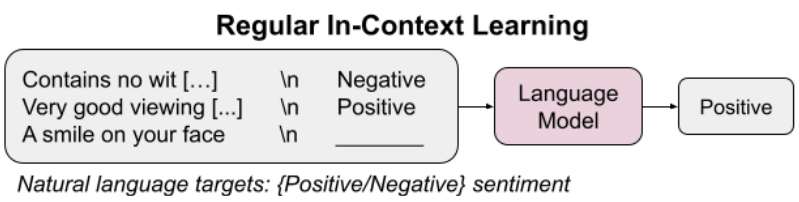
Reminder: Limitation of In-context Learning

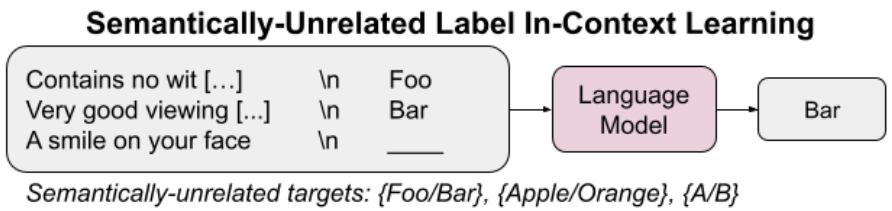
3. Sensitivity to the wording of the prompt, order of examples etc.

No.	Category	Template	Accuracy
1	instructive	Let's think step by step.	**78.7**
2		First, (*1)	77.3
3		Let's think about this logically.	74.5
4		Let's solve this problem by splitting it into steps. (*2)	72.2
5		Let's be realistic and think step by step.	70.8
6		Let's think like a detective step by step.	70.3
7		Let's think	57.5
8		Before we dive into the answer,	55.7
9		The answer is after the proof.	45.7
10	misleading	Don't think. Just feel.	18.8
11		Let's think step by step but reach an incorrect answer.	18.7
12		Let's count the number of "a" in the question.	16.7
13		By using the fact that the earth is round,	9.3

(from lecture 13_14)

4. Lack of clarity regarding what the model learns from the prompt. Even random labels work!



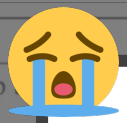


1 Large Models and Tuning

Reminder: Limitation of In-context Learning

3. Sensitivity to the wording of the prompt, order of examples etc.

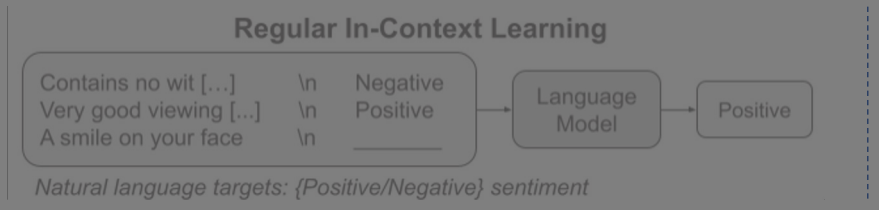
No.	Category	Template	Accuracy
1	instructive	Let's think step	
2		First, (*1)	
3		Let's think about t	
4		Let's solve this pr	
5		Let's be realistic a	
6		Let's think like a c	
7		Let's think	
8		Before we dive int	
9		The answer is after the proof.	45.7
10	misleading	Don't think. Just feel.	18.8
11		Let's think step by step but reach an incorrect answer.	18.7
12		Let's count the number of "a" in the question.	16.7
13		By using the fact that the earth is round,	9.3



In this case, we may need to go back to the **tuning** approach again!

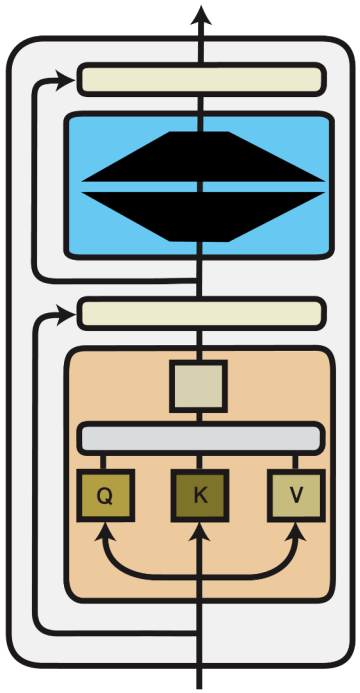
(from lecture 13_14)

4. Lack of clarity regarding what the model learns from the prompt. Even random labels work!



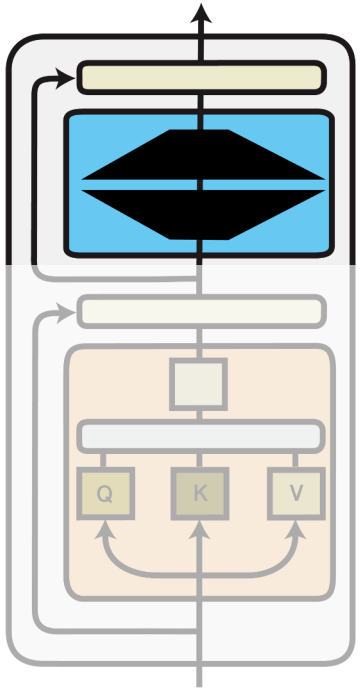
THE UNIVERSITY OF MELBOURNE

From Fine-tuning to Parameter-efficient Fine-tuning



Full Fine-tuning

Update **all model parameters**



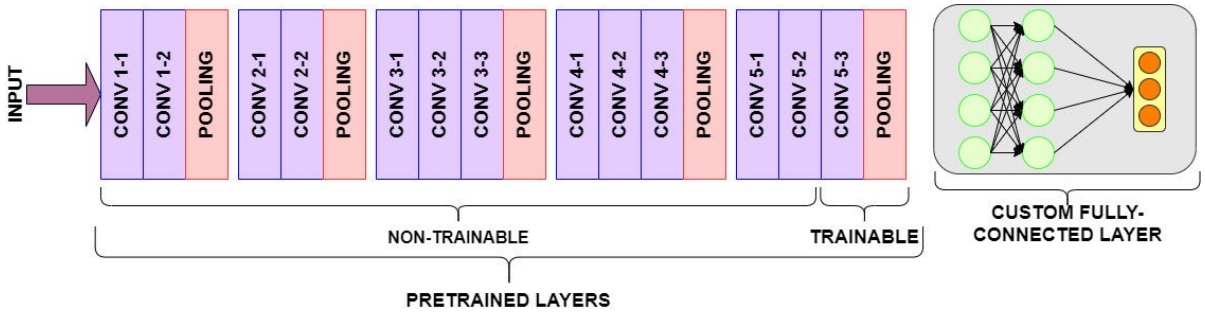
Parameter-efficient Fine-tuning

Update a **small subset** of model parameters

Large Models and Tuning

Any Previous Practice?

- Updating the last layer was common in **computer vision.**



- In NLP, people now have more stacked and over-parameterised models

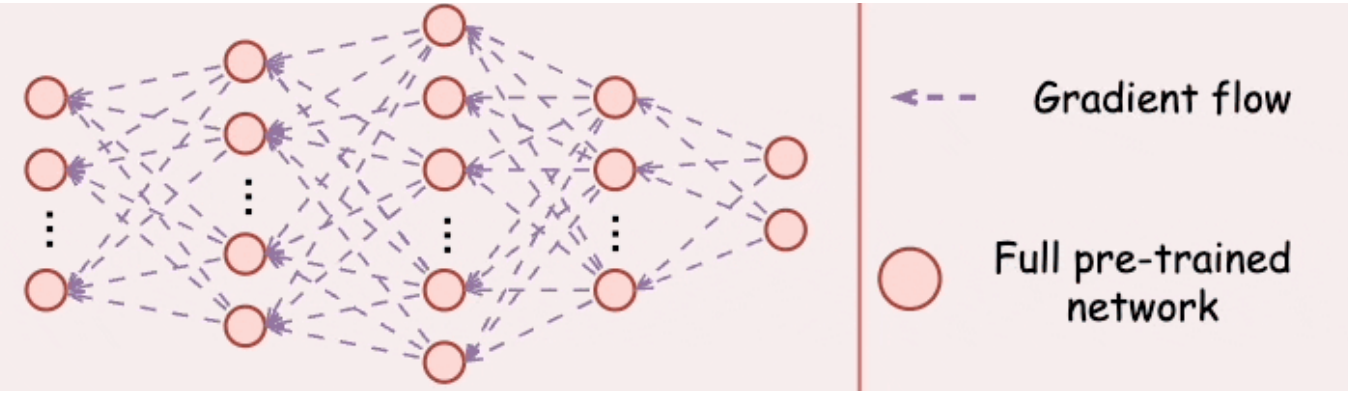
	GPT-1	GPT-2	GPT-3
Training Data	BookCrawl	WebText	CommonCrawl
Parameters	**117M**	**1.5B**	**175B**
Layers	**12**	**48**	**96**
Dimensions	768	1600	12288

“Of course, fine-tuning all parameters performed generally better in practice”

Large Models and Tuning

Then, **why go back** to fine-tuning **only some parameters?**

- Fine-tuning all parameters is impractical with large models



- State-of-the-art models are massively over-parameterized
- **Parameter-efficient fine-tuning usually matches the performance of full fine-tuning**

/ Lecture Plan

THE UNIVERSITY OF MELBOURNE

Lecture 15: Tuning

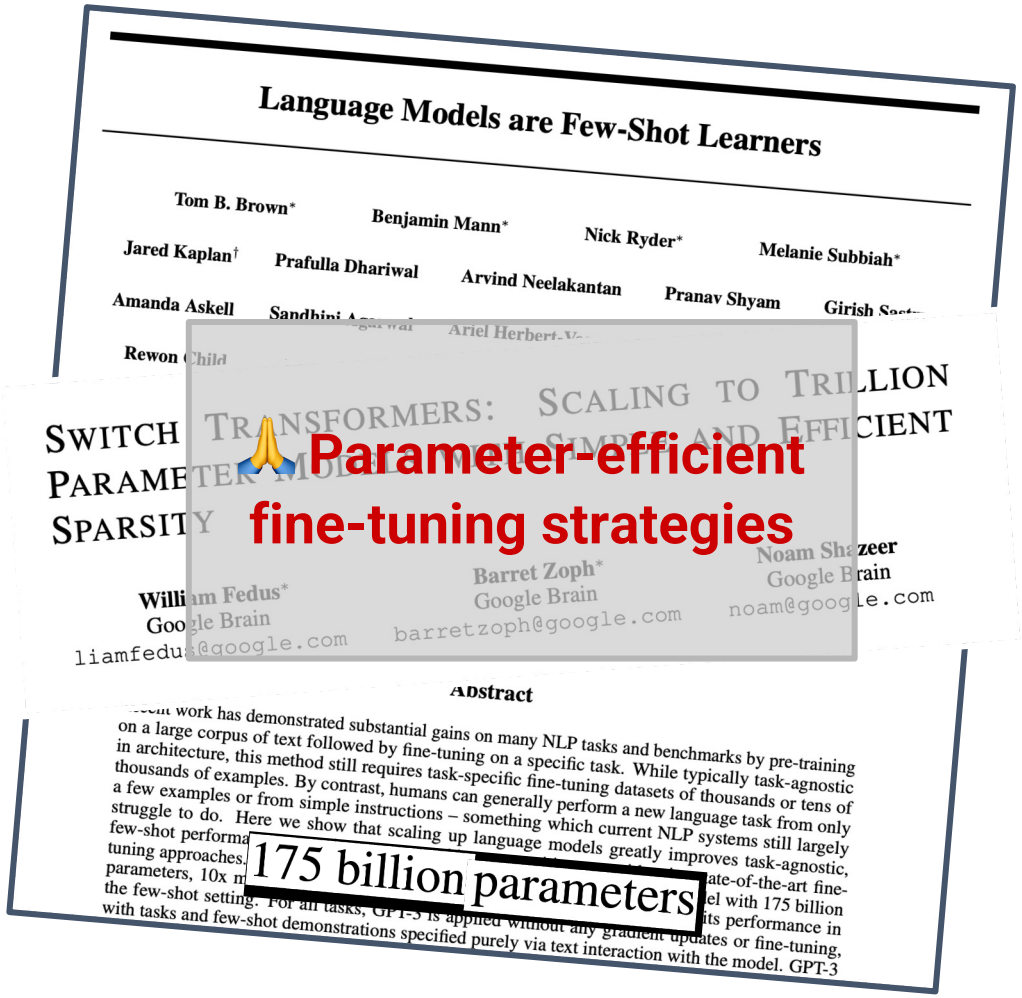
0. In-context Learning vs Fine-Tuning?
0. PLM vs LLM?
1. Large Models and Tuning
2. **Modular and Compositional Representations**
3. Parameter Efficient Models

Modular and Compositional Representations

Modularity and Compositionality

1. Models are **increasing** in **size**

→ Parameter-efficient fine-tuning strategies



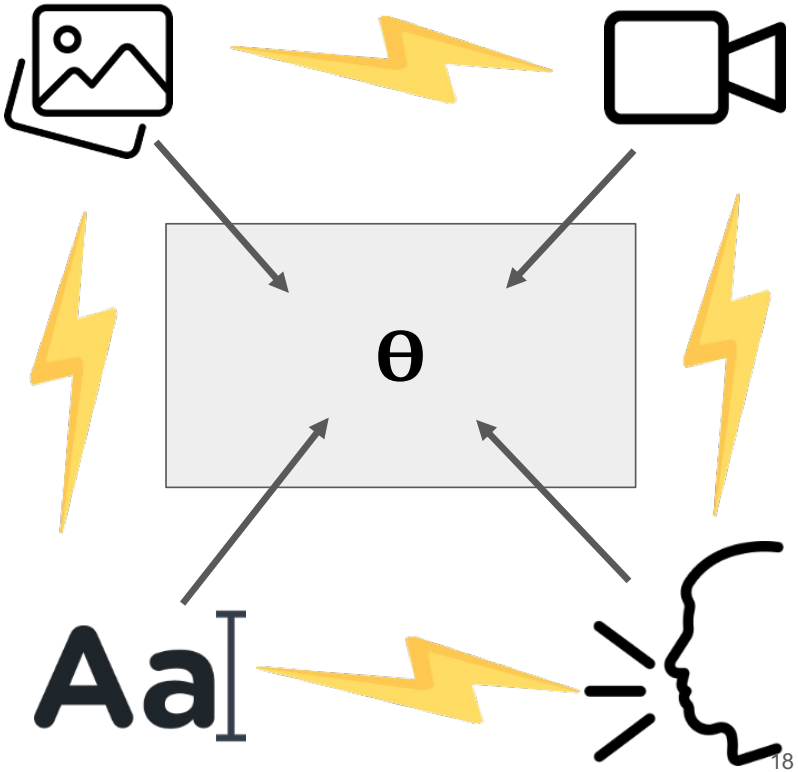
Modular and Compositional Representations

Modularity and Compositionality

1. Models are **increasing** in **size**
→ Parameter-efficient fine-tuning strategies

2. **Unseen** Scenarios
→ Out-of-distribution generalisation through **module composition**

3. Catastrophic interference
→ Modularity as an inductive bias



Modular and Compositional Representations

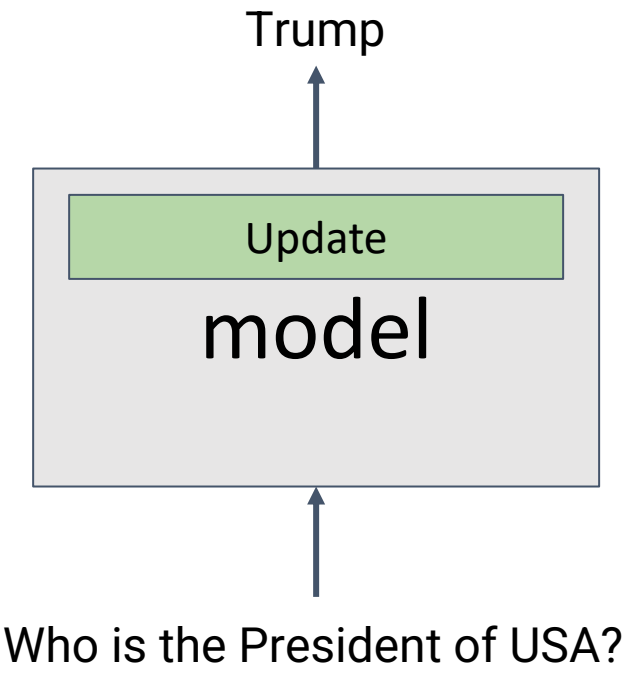
Modularity and Compositionality

1. Models are **increasing** in **size**
→ Parameter-efficient fine-tuning strategies

2. **Unseen** Scenarios
→ Out-of-distribution generalisation through **module composition**

3. Catastrophic interference
→ Modularity as an inductive bias

4. Efficient **updating** of **models** through added components



/ Lecture Plan

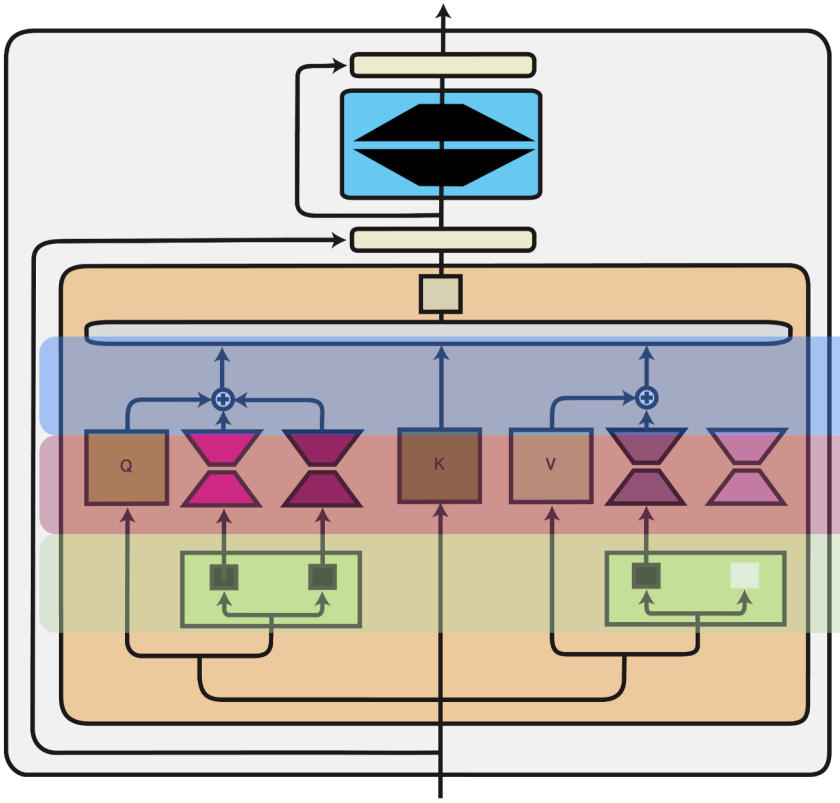
THE UNIVERSITY OF MELBOURNE

Lecture 15: Tuning

0. In-context Learning vs Fine-Tuning?
0. PLM vs LLM?
1. Large Models and Tuning
2. Modular and Compositional Representations
3. **Parameter Efficient Models**

Parameter-Efficient Models

Parameter-Efficient Models



Parameter-efficient Models

Modular Models

Applications

3) Aggregation

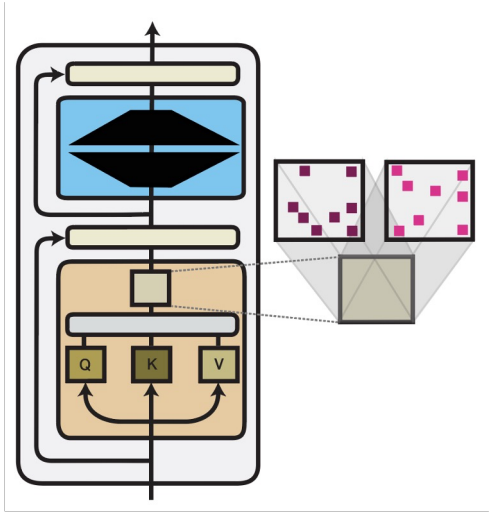
1) Computation Function

2) Routing

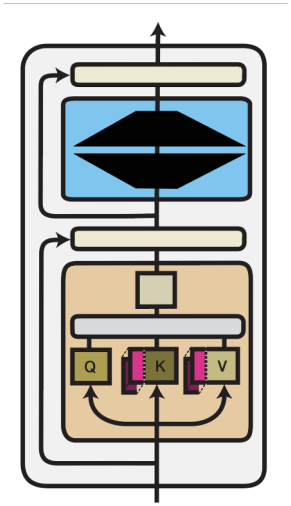
* We will not focus on Routing or Aggregation Variants in this course

Parameter-Efficient Models

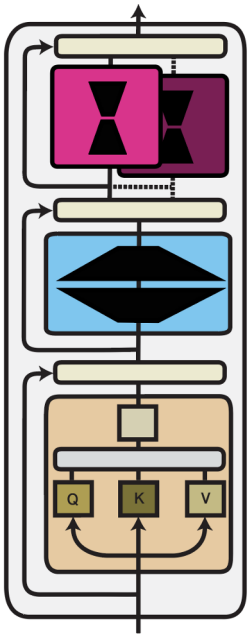
Three Computation Functions



Parameter Composition



Input Composition



Function Composition

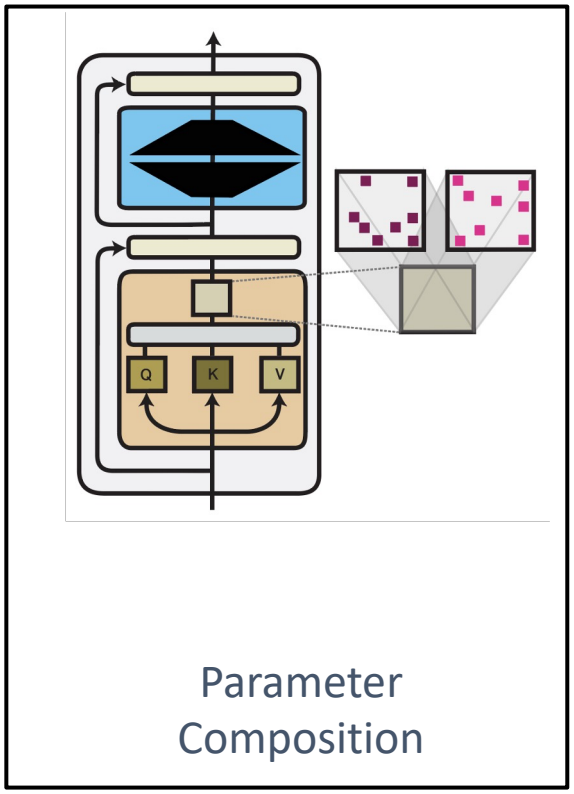
Parameter-Efficient Models

Three Computation Functions

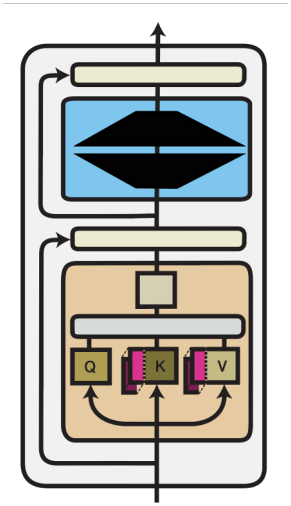
Feature	Parameter Composition	Input Composition	Function Composition
Definition	Efficiently decompose/combine model parameters to reduce size.	Manipulate or combine input data to optimise information representation.	Combine computational functions or layers to build efficient models.
Focus	Efficiency of parameters themselves.	Optimisation of data representation.	Optimisation of the model's computational structure.
Representative Techniques	Low-Rank Decomposition, Sparse Fine-Tuning	Embedding Composition, Prompt tuning	Adapter Layers, Mixture of Experts
Advantages	Reduces the number of parameters and computational cost.	Improves data representation and maintains performance with fewer parameters.	Reduces computational complexity and optimises information flow.

Parameter-Efficient Models

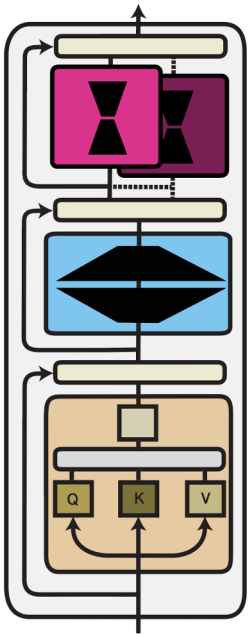
Three Computation Functions



Parameter Composition



Input Composition



Function Composition

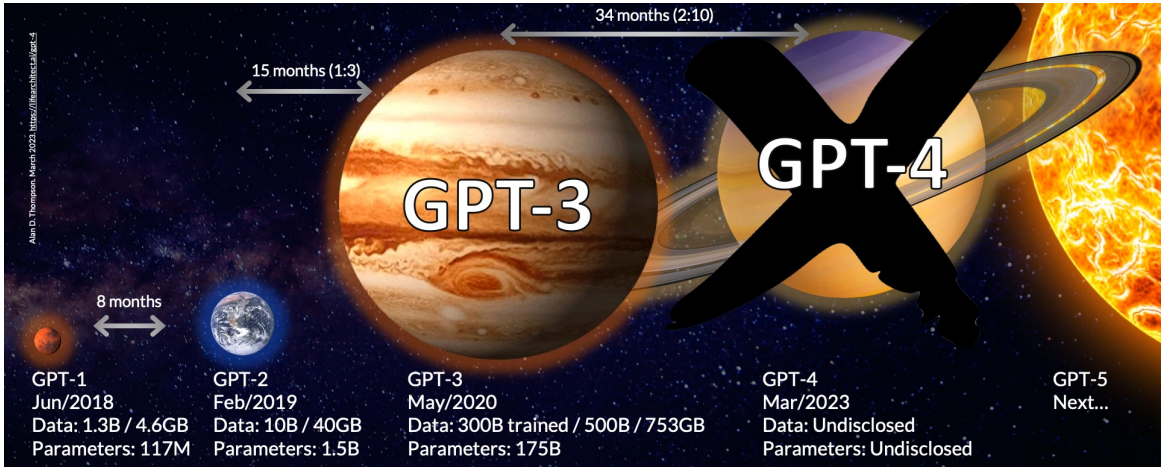
Parameter-Efficient Models

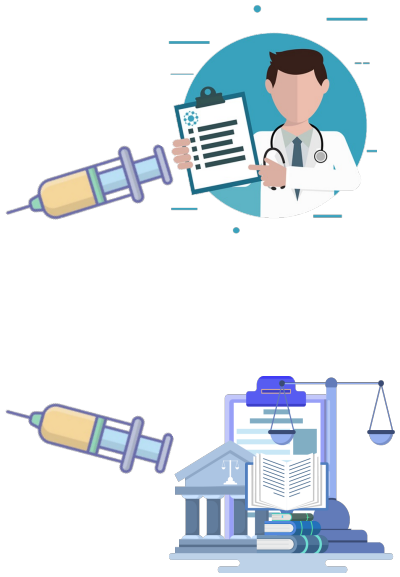
1) Parameter Composition

Best when you want to **apply small updates to a frozen model.**

When? You have limited compute or cannot fine-tune the whole model.

- Adapting an LLM (e.g., LLaMA, GPT) to a specific domain like medical or legal texts.
- Efficient fine-tuning with minimal trainable parameters.

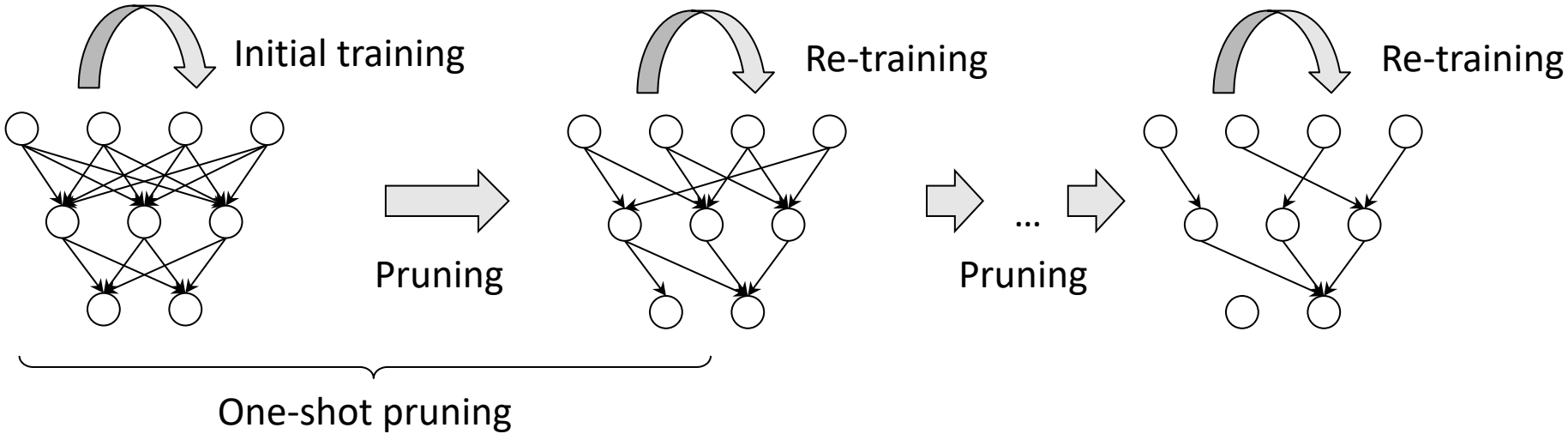




Parameter Composition

Sparse Subnetworks

1. A common inductive bias on the module parameters is **sparsity**
2. Most common sparsity method: **pruning**
3. Pruning can be seen as applying a **binary mask** that selectively keeps or removes each connection in a model and produces a subnetwork.

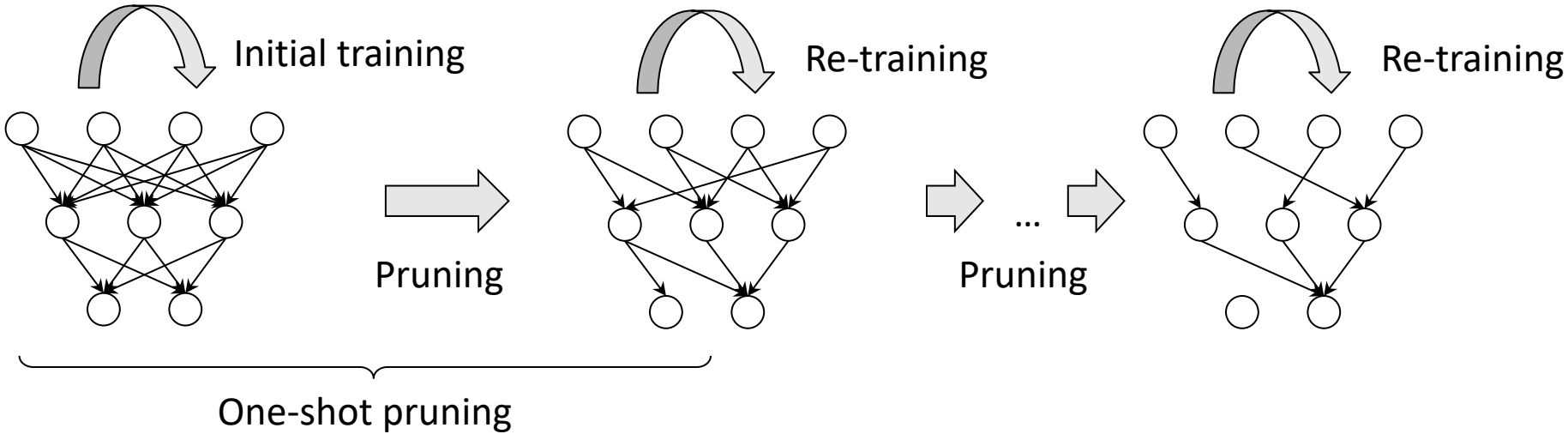


Parameter Composition

THE UNIVERSITY OF MELBOURNE

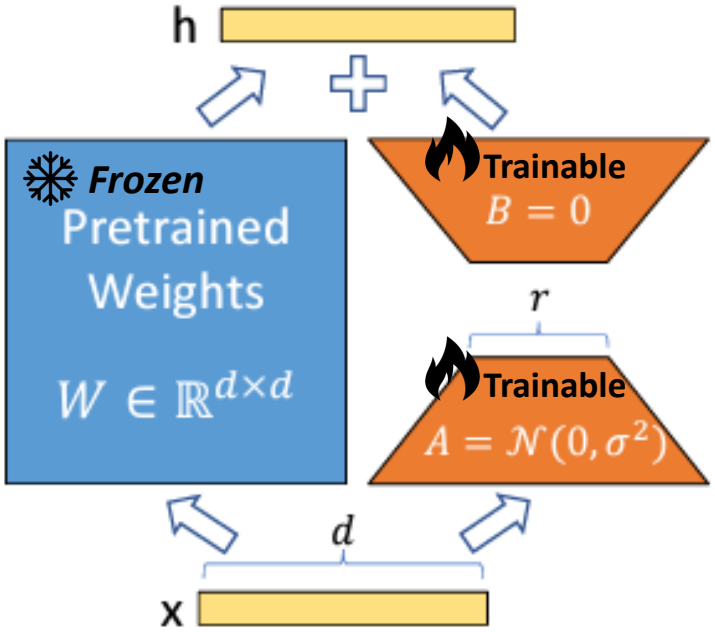
Sparse Subnetworks

1. During pruning, a fraction of the lowest-magnitude weights are removed
2. The non-pruned weights are re-trained
3. Pruning for multiple iterations is more common



Parameter-Efficient Models

Low-Rank Adaptation (LoRA)



LoRA

Core Concept

Fine-tuning large models is expensive

- Do not update full weight matrix (W)
- Instead, update only the **change** (ΔW):

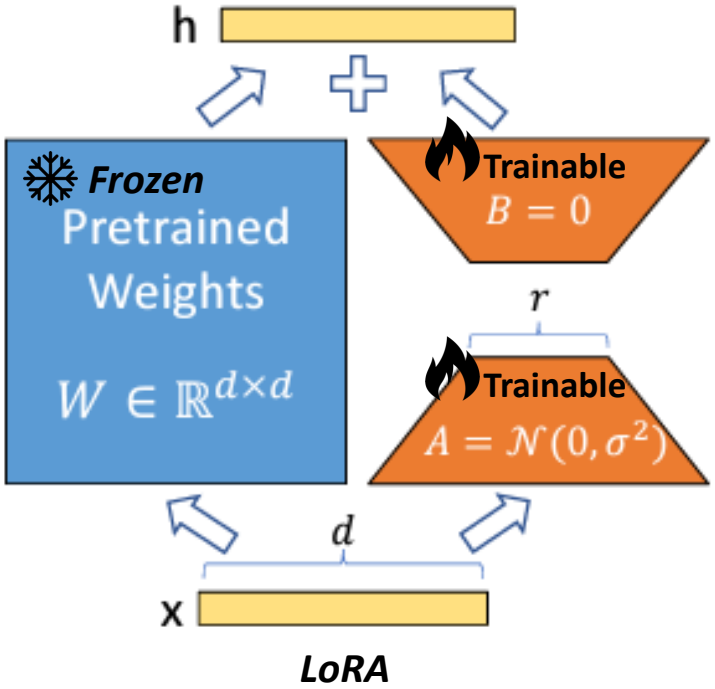
$$W' = W + \Delta W$$

- Approximate as product of two small matrices:

$$\Delta W \approx B \cdot A$$

Parameter Composition

Low-Rank Adaptation (LoRA)



LoRA

What is low rank means

- W is a large full-rank matrix
- LoRA only learns a low-rank change

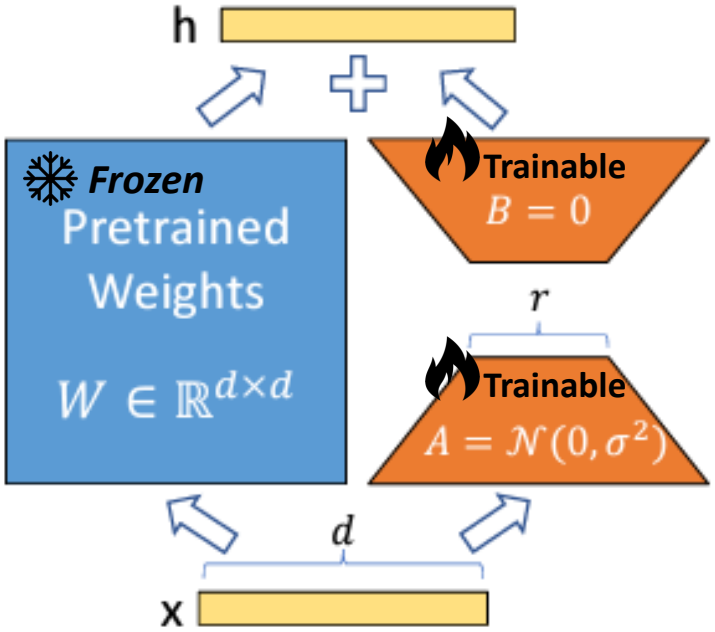
$$\Delta W \approx B \cdot A$$

$$A \in \mathbb{R}^{r \times d},\ B \in \mathbb{R}^{d \times r} \Rightarrow \mathrm{rank}(\Delta W) \leq r$$

"Low-Rank"
→ compressed representation of change
(r is small)

Parameter Composition

Low-Rank Adaptation (LoRA)



LoRA

Why it is efficient?

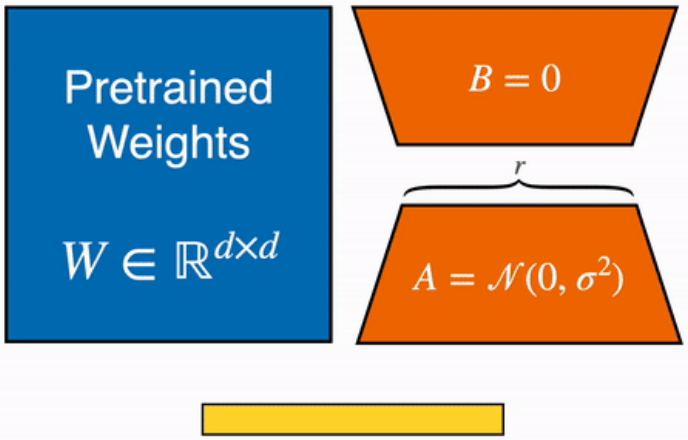
- Suppose W: 1024 x 1024 (1M params)
- LoRA uses A (r x d), B (d x r), where r << d (r is *much smaller* than d)

For example (d=1024, r = 8),

- **A**: 8 x 1024, **B**: 1024 x 8
- Total trainable: 2 x 1024 x 8 = 16,384
- Reduction: 98.4% fewer parameters to update!

Parameter Composition

Low-Rank Composition



LoRA

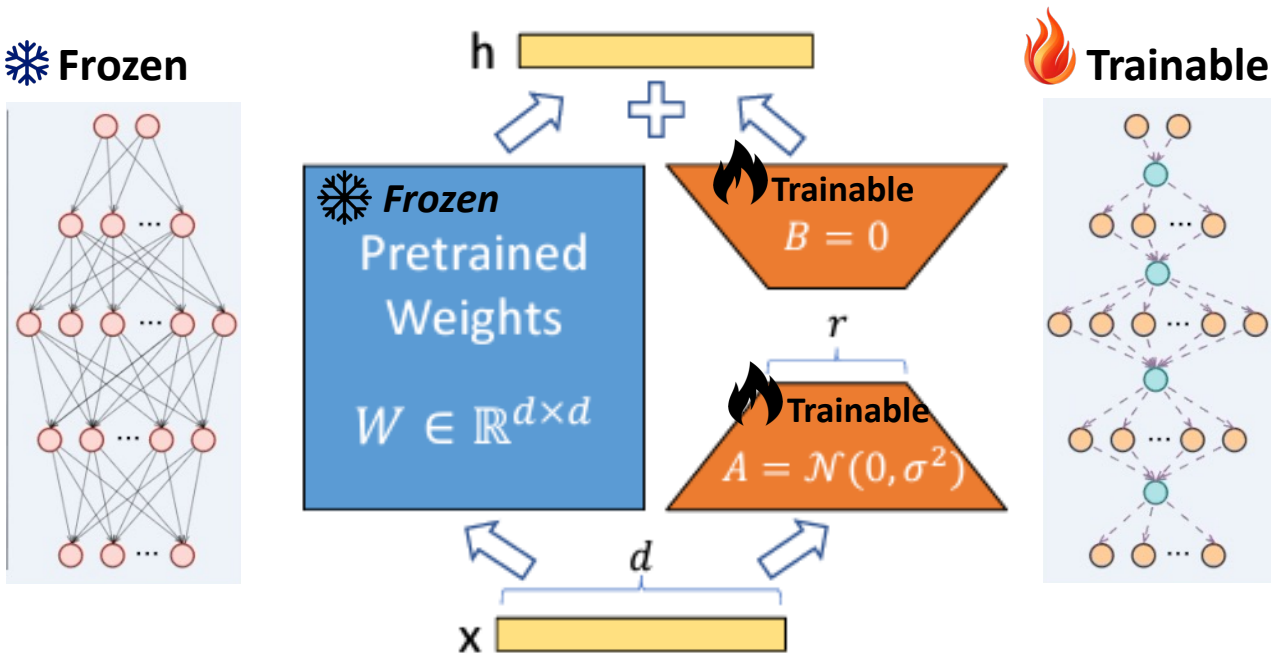
What is the Output

$$\text{Output} = (W + B \cdot A) \cdot x = W \cdot x + B \cdot (A \cdot x)$$

- W is frozen
- Only A and B are trained
- x is the input vector to the layer — typically a token embedding or the previous layer's output

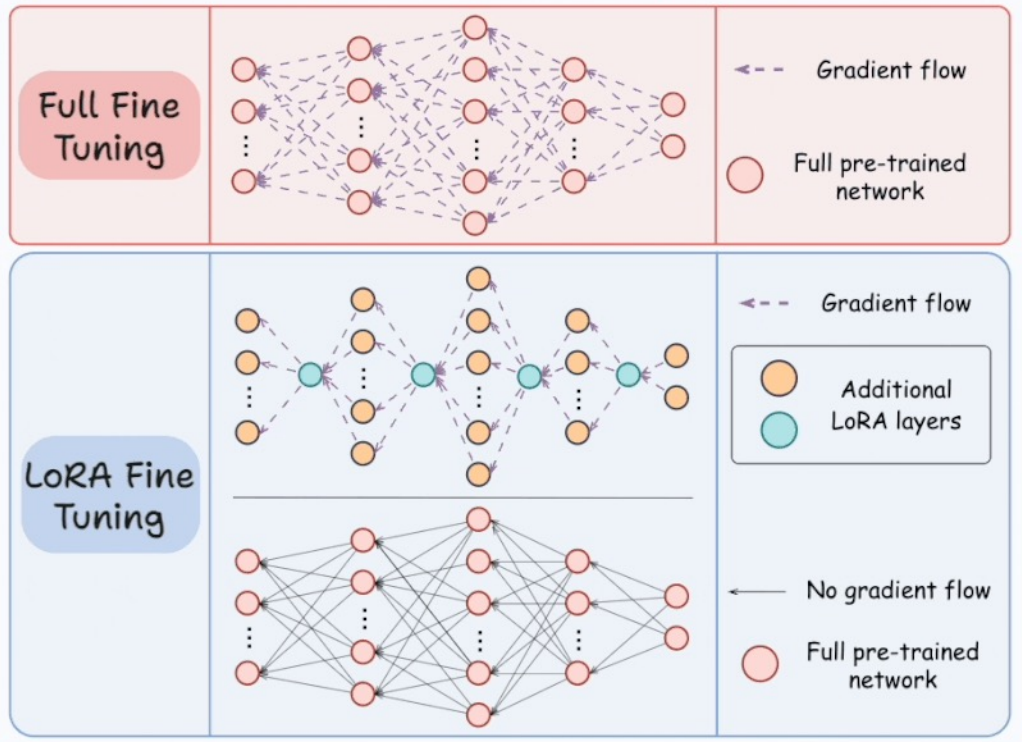
Parameter Composition

Low-Rank Composition - Backpropagation



Parameter Composition

Low-Rank Composition



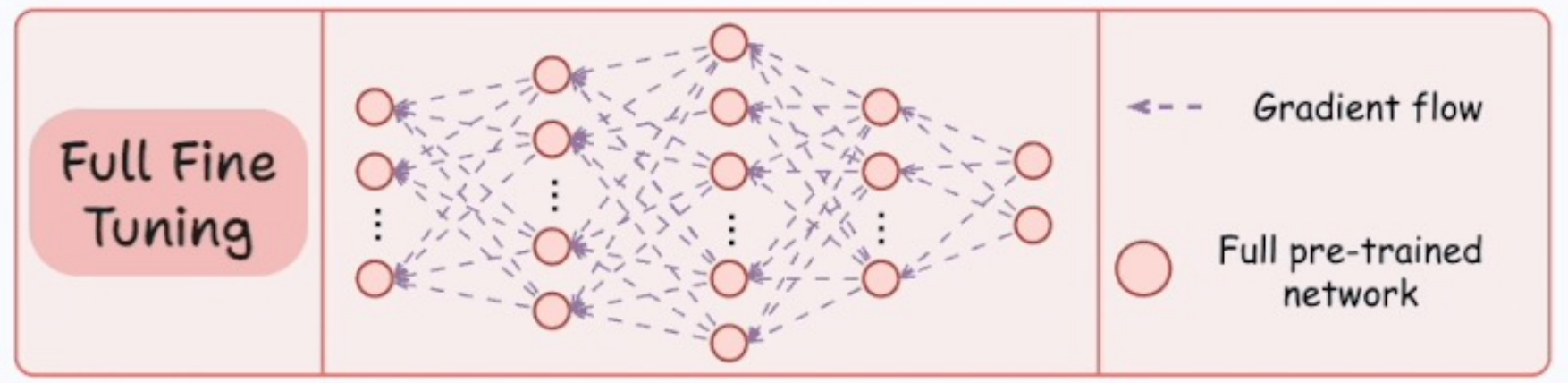
Low-Rank Adaptation (LoRA)

- *Train A and B only instead of W*
- *Do not introduce additional inference latency*
- *Up to 2/3 reduction in VRAM usage, 25% speed up in training*

Parameter Composition

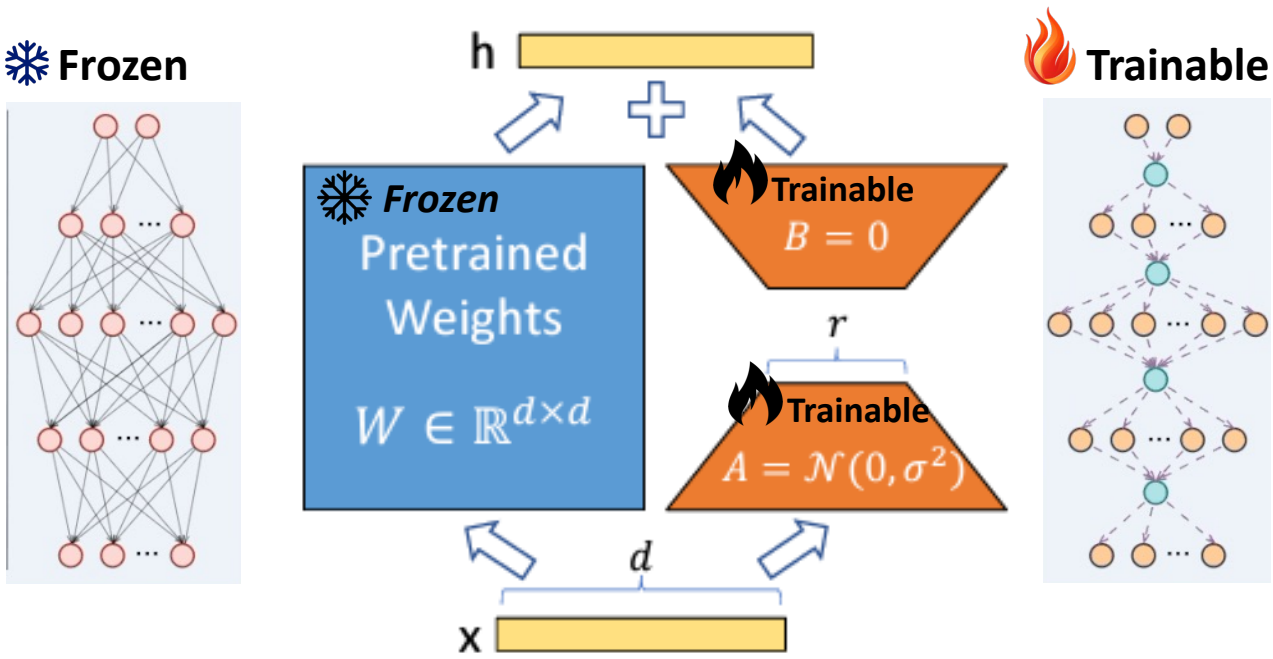
THE UNIVERSITY OF MELBOURNE

Full Tuning - Backpropagation



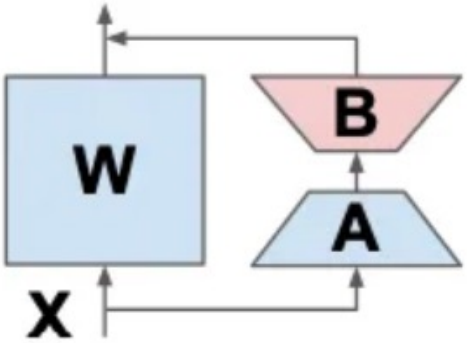
Parameter Composition

Low-Rank Composition - Backpropagation



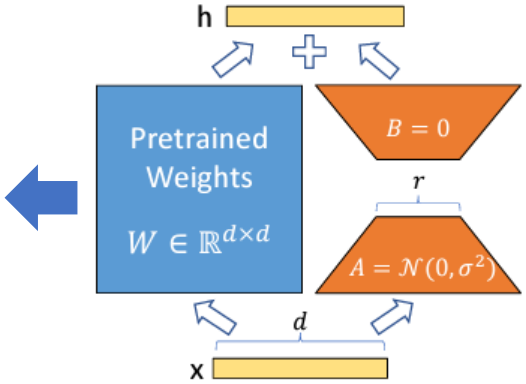
Parameter Composition

LoRA and its variants

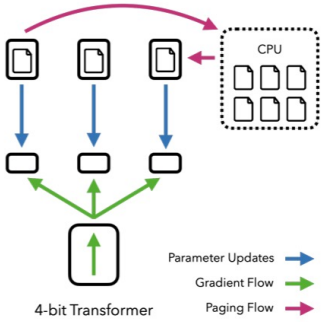


LoRA-FA

Allows adaptation of more parts of the model (e.g., biases or layer norms).

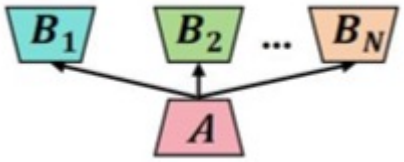


LoRA



QLoRA

Combines **4-bit quantisation** of the base model with **LoRA fine-tuning**.



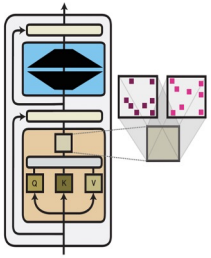
HydraLoRA

Loads and switches between **multiple LoRA adapters**

Parameter Composition

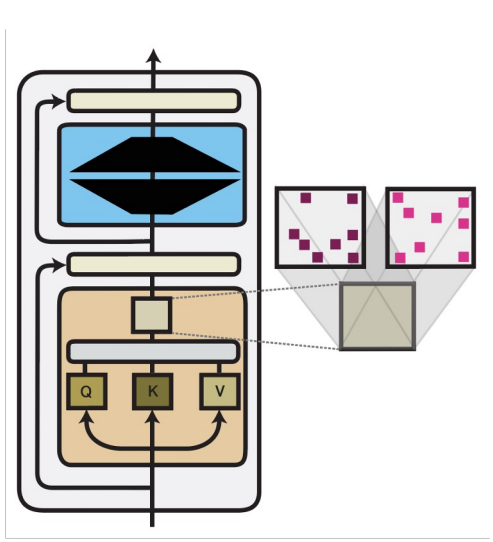
THE UNIVERSITY OF MELBOURNE

Computation Functions Comparison

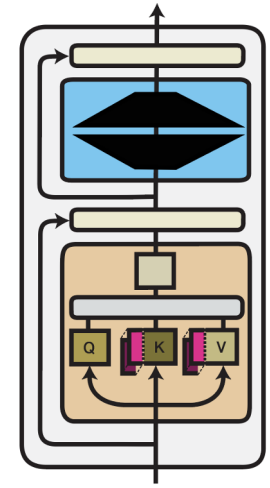
		Parameter efficiency	Training efficiency	Inference efficiency	Performance	Composition-ality
Parameter composition		Methods require < 0.5% of parameters	Pruning requires re-training iterations	Does not increase the model size	E.g., LoRA achieves strong performance	Subnetworks can be composed
		+	-	++	+	+

Parameter-Efficient Models

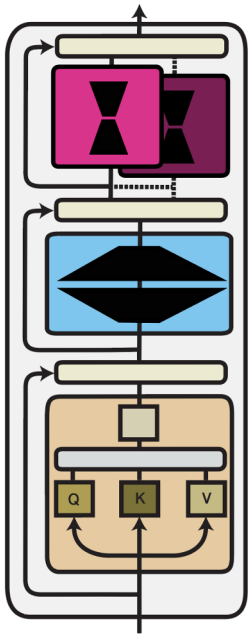
Three Computation Functions



Parameter Composition



Input Composition



Function Composition

Augment a model's input by augmenting it with a learnable parameter vector

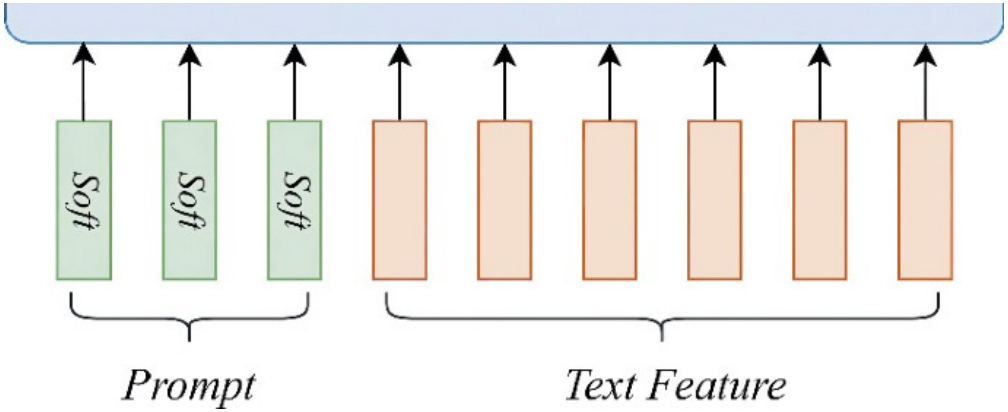
Input Composition

2) Input Composition

Ideal when the **model must remain unchanged** and **only inputs can be modified.**

When? You use a frozen model where weights can't be updated

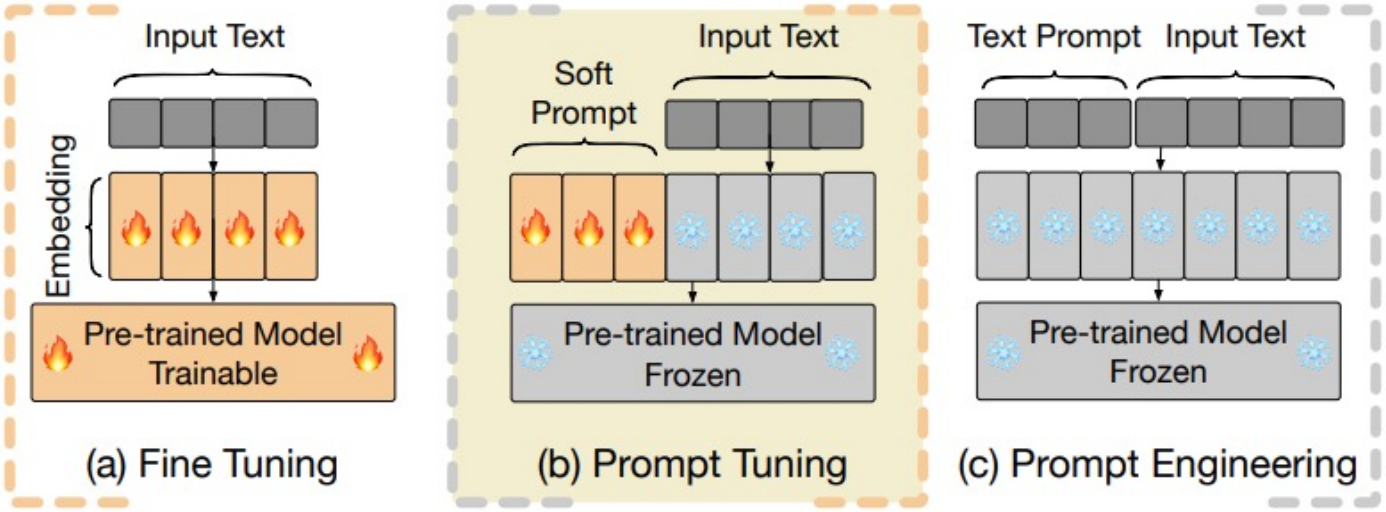
- Prompt tuning, prefix injection, or few-shot learning via task-specific input templates.
- Learn a small set of soft prompt embeddings that are prepended to the input sequence.



Input Composition

Input Composition and Prompt

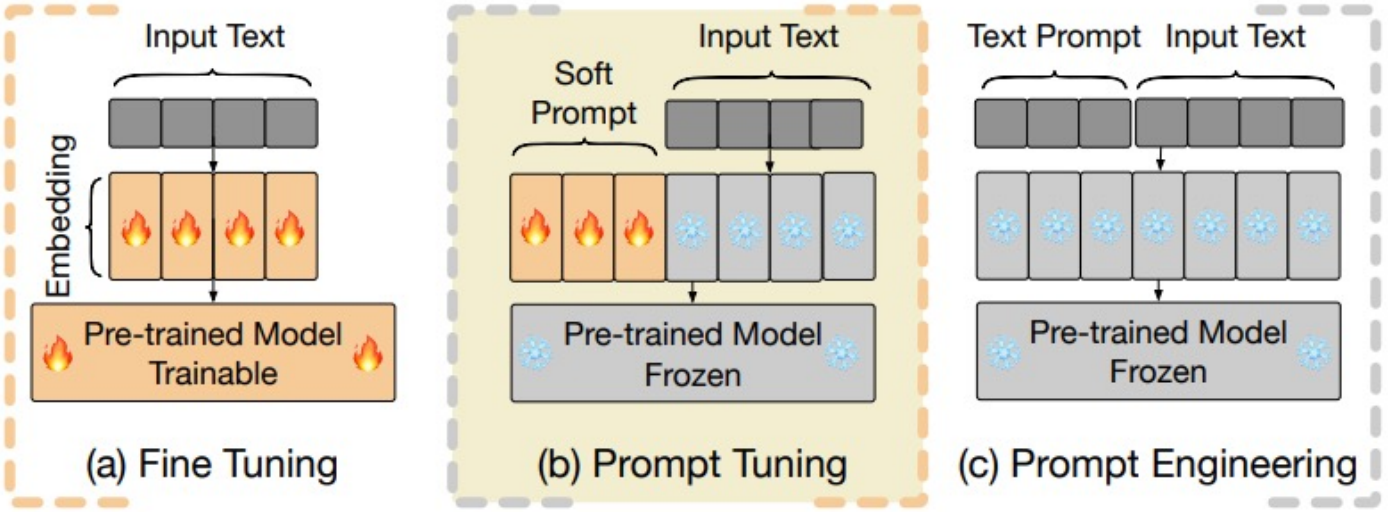
1. Input Composition = adapting the model by changing the input, not the model.
2. Fine-Tuning updates **model parameters**, which is not input composition
3. Prompt Tuning uses **learned soft prompts**, while Prompt Engineering relies on **manual text prompts**.



Input Composition

Input Composition and Prompt

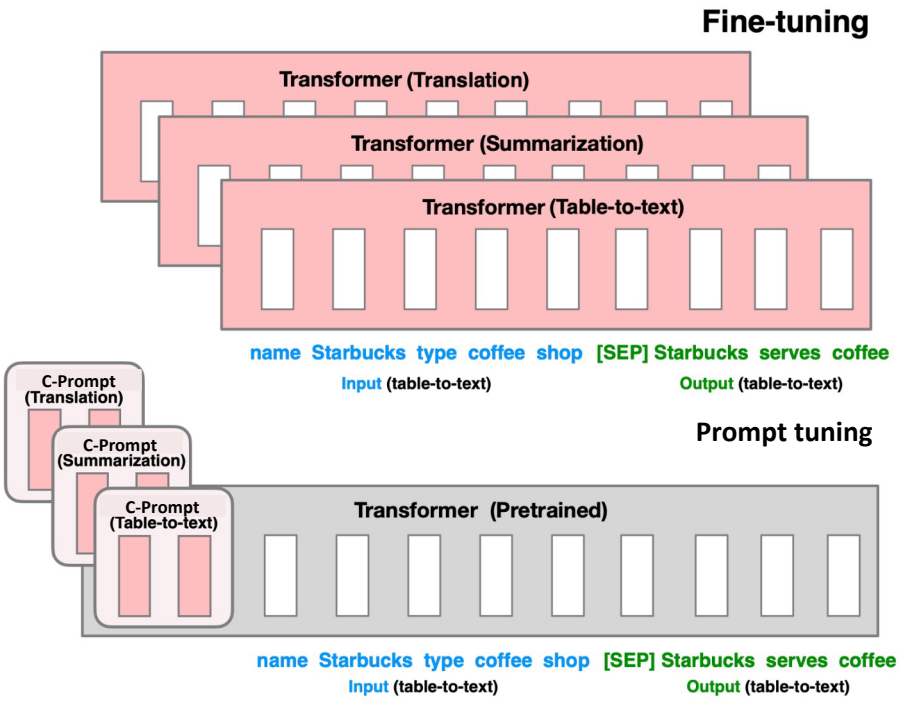
Method	Resource	Training	Best For
Fine-Tuning	High	Yes	Tasks requiring deep model customization
Prompt Tuning	Low	Yes	Maintaining model integrity across tasks
Prompt Engineering	None	No	Quick adaptations with no computational cost



Input Composition

Prompt Tuning

1. Directly learn a continuous prompt that is prepended to the input.
2. Typically, a matrix consisting of a sequence of continuous prompt embeddings

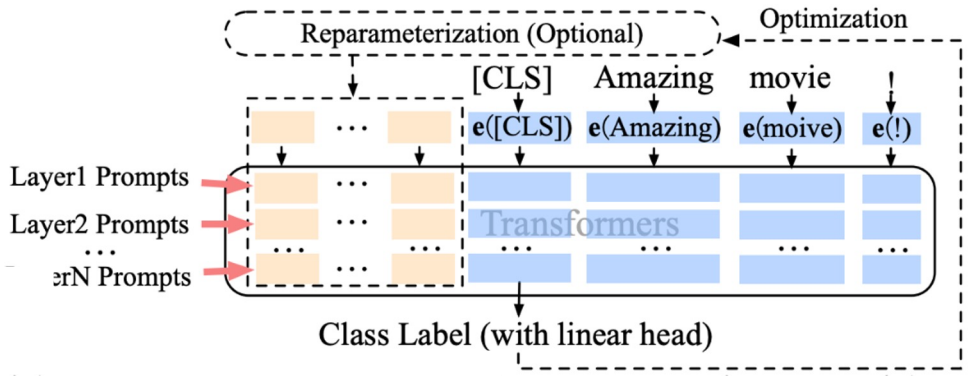


[Soft prompt embedding] +
[Original input embedding]

Input Composition

Multi-Layer Prompt Tuning

1. Instead of learning parameters only at the input layer, we can learn them at every layer of the model
2. Continuous prompts in later layers are more important (In practice, continuous prompts are concatenated with the keys and values in the self-attention layer)

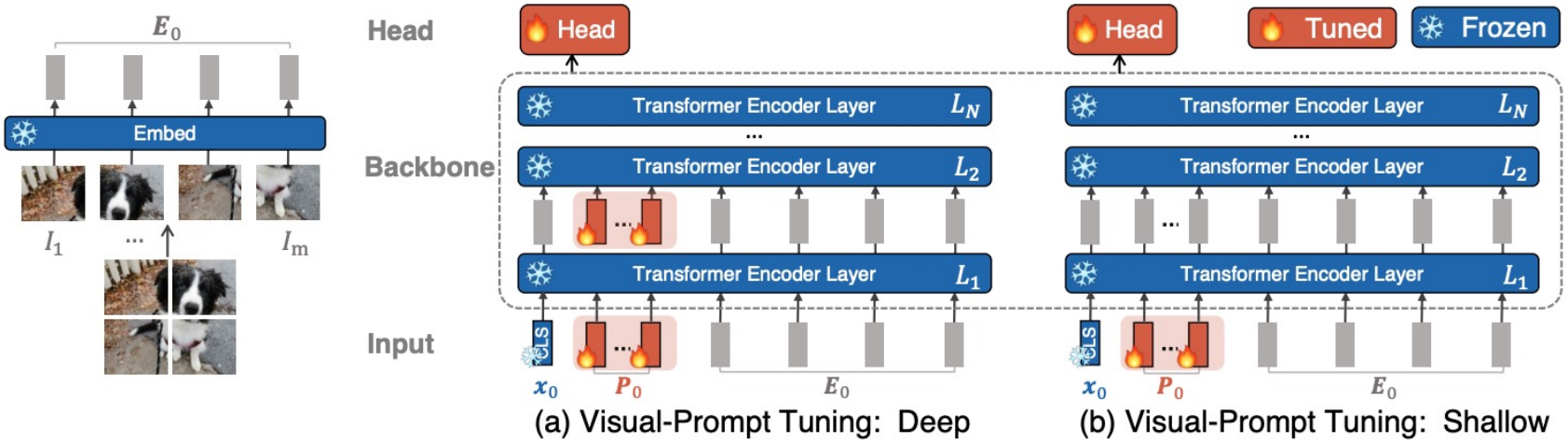


Multi-layer prompt tuning

Input Composition

Multi-Layer Prompt Tuning

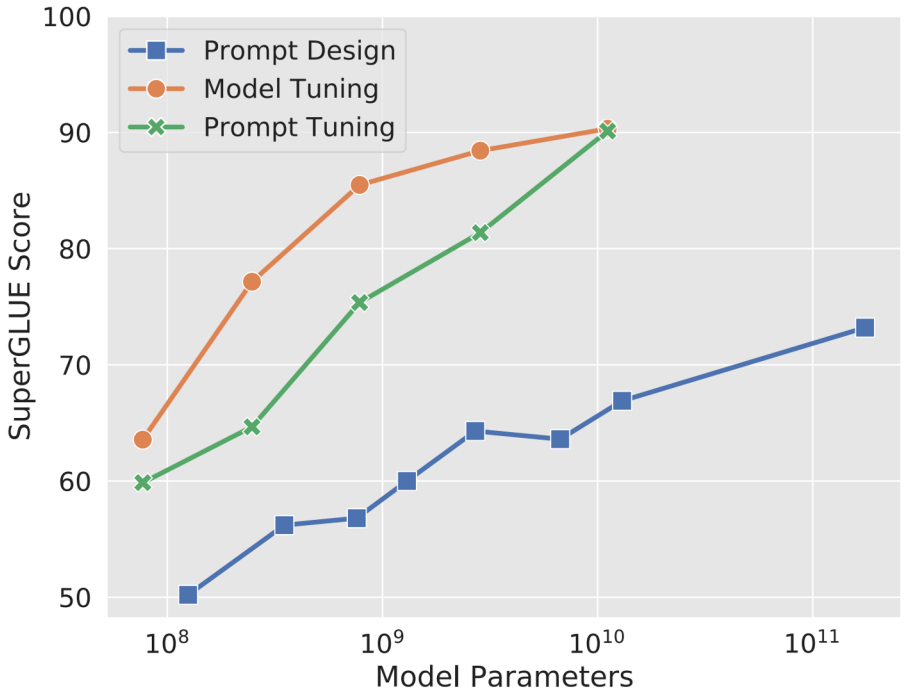
Visual-Prompt Tuning (VPT) for adapting transformer models in computer vision. Instead of training the whole model, they prepend prompts to the input of the model and train the prompts and head.



Input Composition

Prompt Tuning Only Works Well at Scale

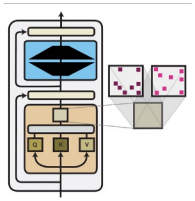
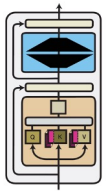
Prompt tuning performs poorly at smaller model sizes and on harder tasks.



Prompt tuning vs standard fine-tuning and prompt design across T5 models of different sizes

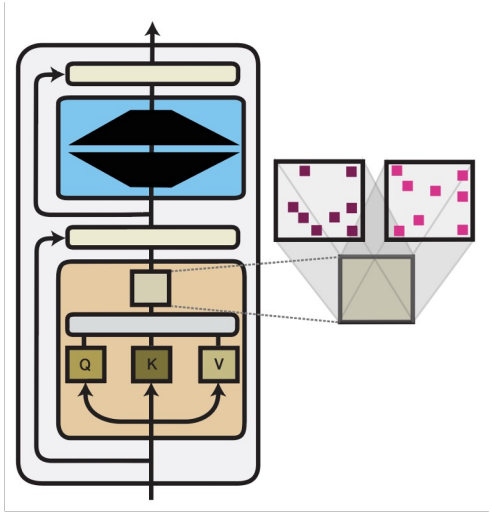
Parameter-Efficient Models

Computation Functions Comparison

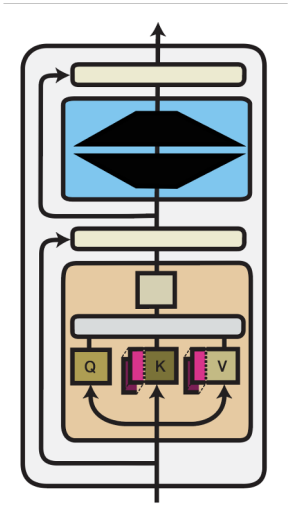
		Parameter efficiency	Training efficiency	Inference efficiency	Performance	Composition-ality
Parameter composition		Methods require < 0.5% of parameters	Pruning requires re-training iterations	Does not increase the model size	E.g., LoRA achieves strong performance	Subnetworks can be composed
		+	-	++	+	+
Input composition		Only add a small number of parameters	Extend the model's context window		Requires large models to perform well	Continuous prompts have been composed
		++	--	--	-	+

Parameter-Efficient Models

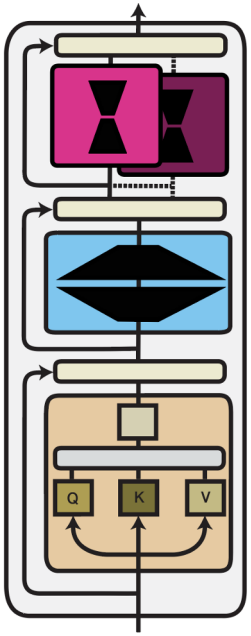
Three Computation Functions



Parameter Composition



Input Composition



Function Composition

Function Composition

THE UNIVERSITY OF MELBOURNE

3) Function Composition

Useful when **combining multiple skills, domains, or adapters logically.**

When? You need to switch or combine multiple **adapters** for multi-task or multi-domain.

- Merging a “legal-domain adapter” and an “emotion-detection adapter” in PEFT.
- Flexible and modular control over model behaviour.

- Function composition augments a model’s functions with new task-specific functions:

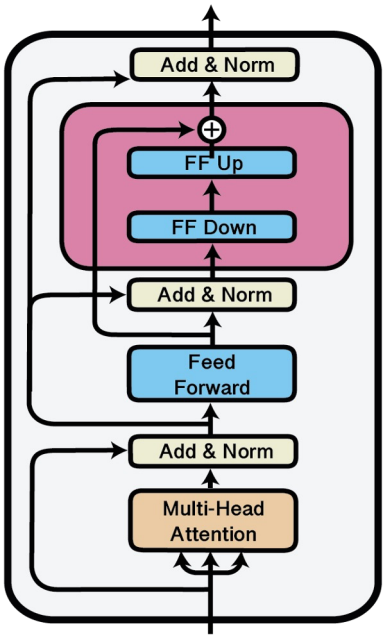
$$f_i'(\boldsymbol{x}) = f_{\theta_i}(\boldsymbol{x}) \odot f_{\phi_i}(\boldsymbol{x})$$

Function Composition

THE UNIVERSITY OF MELBOURNE

Adapters

1. The main purpose of functions added to a pre-trained model is to adapt it
2. The design of adapters is model-specific
3. In NLP, an adapter in a Transformer layer typically consists of a **feed-forward down-projection**, a **feed-forward up-projection** and an **activation function**
4. The adapter is usually placed after the multi-head attention and/or after the feed-forward layer



Function Composition

IA^3

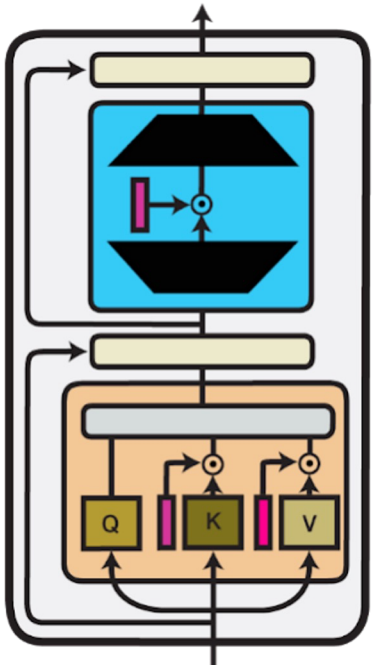
IA^3 multiplies learned vectors with the **keys** and **values** in self-attention and the **intermediate activations** in the feed-forward network of a Transformer

A Transformer block typically consists of:

- **Multi-head Attention**:
  $Q = XW^Q, \quad K = XW^K, \quad V = XW^V$
- **Feedforward Network (FFN)**:
  $FFN(X) = W_2(\mathrm{ReLU}(W_1 X))$

IA^3 introduces **three learnable vectors**:

1. $\lambda_q \in \mathbb{R}^d$ — scales the **Query**
2. $\lambda_k \in \mathbb{R}^d$ — scales the **Key**
3. $\lambda_f \in \mathbb{R}^d$ — scales the **FFN output**



IA^3 in the Transformer

Function Composition

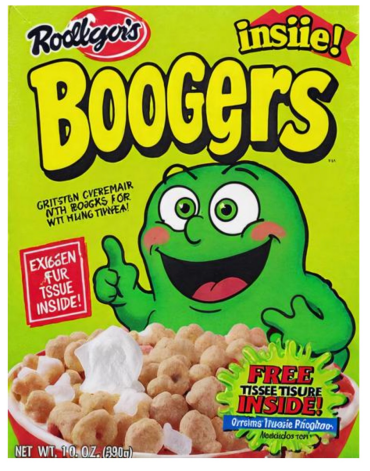
LoRA as an Adapter

LoRA originally used Parameter Composition,
but when using multiple adapters, it can be combined in Function Composition for more flexibility.

sushi rolls shaped like kawaii cat faces

LLM (text-to-image generation)

Super Cereal
- SDXL LoRA



https://huggingface.co/docs/peft/main/en/tutorial/peft_integrations

Function Composition

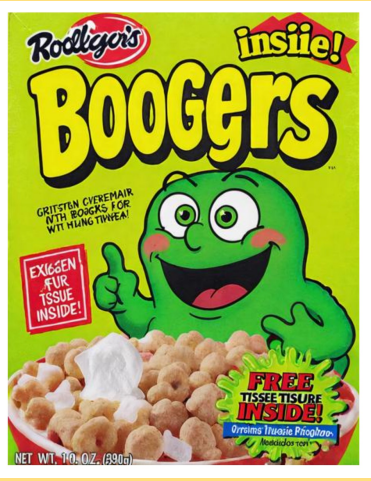
LoRA as an Adapter

LoRA originally used Parameter Composition,
but when using multiple adapters, it can be combined in Function Composition for more flexibility.

sushi rolls shaped like kawaii cat faces

LLM (text-to-image generation)

Super Cereal
- SDXL LoRA



sushi rolls shaped like kawaii cat faces

LLM (text-to-image generation)

https://huggingface.co/docs/peft/main/en/tutorial/peft_integrations

Function Composition

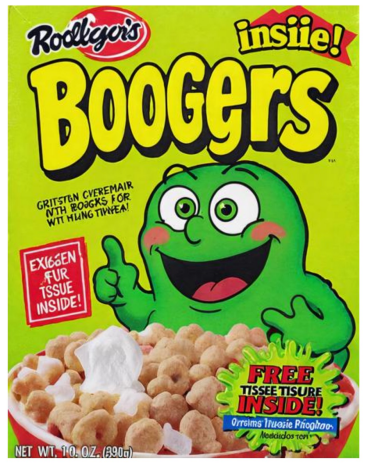
LoRA as an Adapter

LoRA originally used Parameter Composition,
but when using multiple adapters, it can be combined in Function Composition for more flexibility.

sushi rolls shaped like kawaii cat faces

LLM (text-to-image generation)

Super Cereal
- SDXL LoRA



sushi rolls shaped like kawaii cat faces

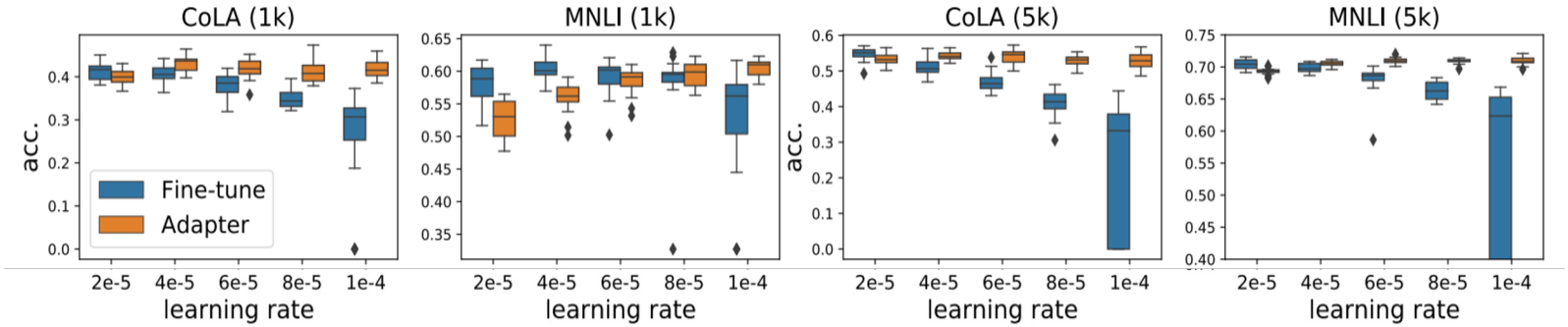
LLM (text-to-image generation)

https://huggingface.co/docs/peft/main/en/tutorial/peft_integrations

Function Composition

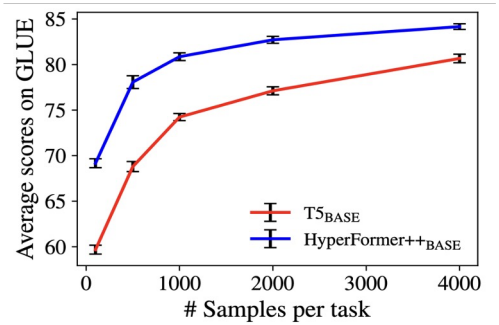
Benefits of Adapters

1. Increased robustness



BERT test performance distributions over 20 runs with different learning rates

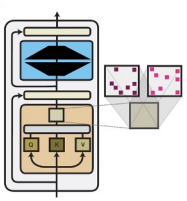
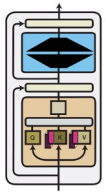
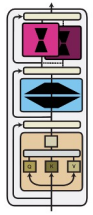
2. Increased sample efficiency



Results on GLUE with different numbers of training samples per task

Parameter-Efficient Models

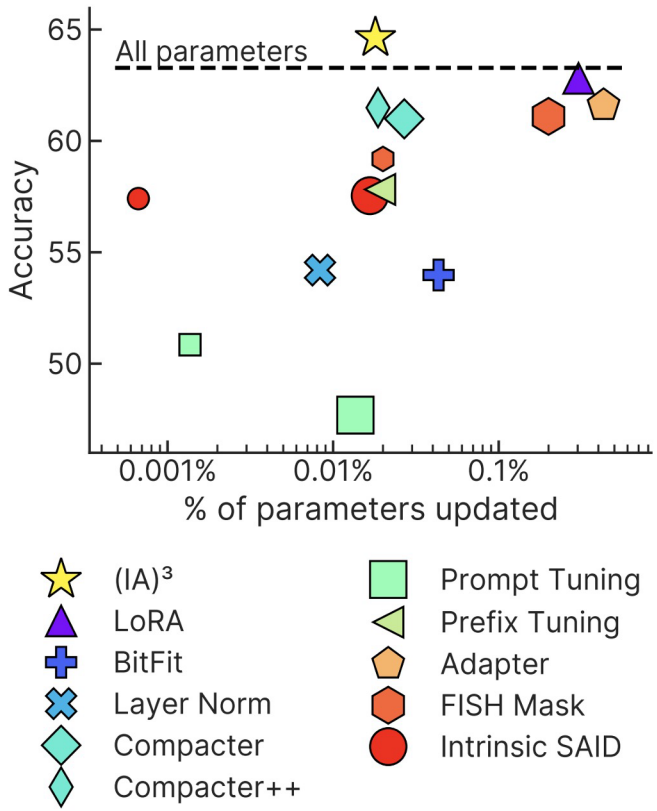
Computation Functions Comparison

		Parameter efficiency	Training efficiency	Inference efficiency	Performance	Composition-ality
Parameter composition		Methods require < 0.5% of parameters	Pruning requires re-training iterations	Does not increase the model size	E.g., LoRA achieves strong performance	Subnetworks can be composed
		+	-	++	+	+
Input composition		Only add a small number of parameters	Extend the model's context window		Requires large models to perform well	Continuous prompts have been composed
		++	--	--	-	+
Function Composition		**Adapters depend on the hidden size**	**Does not requir e gradients of frozen params**	**New functions increase # of operations**	**Match or outperform standard fine-t uning**	**Adapters can be composed**
		-	+	-	++	+

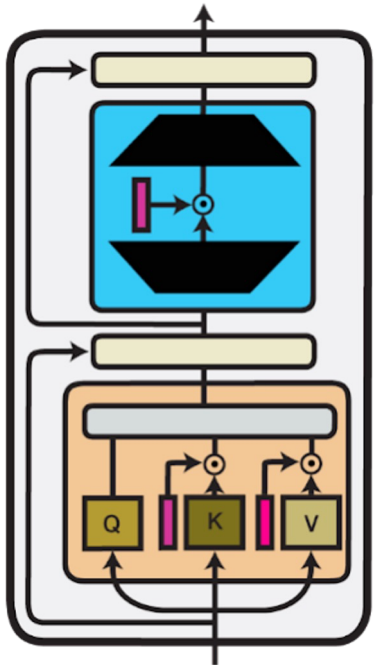
Parameter-Efficient Models

Performance Comparison

Function composition methods such as adapter, compacter, and IA^3 achieve the best performance but add more parameters



IA^3



Parameter-Efficient Models

Which type of PEFT is better?
1) Input Composition, **2)** Parameter Composition, **3)** Function Composition

1. Case: Build a **biomedical** Q&A system, where the **LLM** must retrieve passages and logically **combine** evidence across documents (e.g., chain-of-thought **reasoning** over retrieved facts).

[redacted]

2. Case: You use an **open-source LLM** and want to adapt it to a new **legal** domain with **minimal training**. You have enough access to inject or fine-tune certain parts of the model, but want to avoid retraining everything.

[redacted]

Parameter-Efficient Models

Which type of PEFT is better?
1) Input Composition, **2)** Parameter Composition, **3)** Function Composition

3. Case: You're working with a large frozen model (e.g., API) to build a customer support chatbot. You cannot modify the model weights or architecture, but you want to tailor the model's behaviour to your company's tone and domain.

[redacted]

4. Case: You have fine-tuned a base LLM for three different tasks (e.g., summarisation, classification, translation). You want to dynamically switch between tasks at inference time **without duplicating the full model** each time.

[redacted]